Model Question Paper-I/II with effect from 2022-23 (CBCS Scheme)

USN | | | | | | | | |

First/Second Semester B.E. Degree Examination PRINCIPLES OF PROGRAMMING USING C

TIME: 03 Hours Max. Marks: 100

Note: 01. Answer any FIVE full questions, choosing at least ONE question from each MODULE. 02. Use C code snippet to illustrate a specific code design or a purpose

Module -1			*Bloom's Taxonomy Level	Marks

Q.01	a	**Explain the structure of C program in detail. Write a sample program to demonstrate the components in the structure of C program**
		The basic structure of a C program is divided into 6 parts which makes it easy to read, modify, document, and understand in a particular format. C program must follow the below-mentioned outline in order to successfully compile and execute. Debugging is easier in a well-structured C program.
		Sections of the C Program
		There are 6 basic sections responsible for the proper execution of a program. Sections are mentioned below:
		1. Documentation
		2. Preprocessor Section
		3. Definition
		4. Global Declaration
		5. Main() Function
		a. Sub Programs
		1. Documentation
		This section consists of the description of the program, the name of the program, and the creation date and time of the program. It is specified at the start of the program in the form of comments. Documentation can be represented as:
		`// description, name of the program, programmer name, date, time etc.`
		or
		`/*`
		`description, name of the program, programmer name, date, time etc.`
		`*/`
		Anything written as comments will be treated as documentation of the
			L2	8

vtuedge.com

program and this will not interfere with the given code. Basically, it gives an overview to the reader of the program.

2. Preprocessor Section

All the header files of the program will be declared in the preprocessor section of the program. Header files help us to access other's improved code into our code. A copy of these multiple files is inserted into our program before the process of compilation.

Example:

```
#include<stdio.h>
#include<math.h>
```

3. Definition

Preprocessors are the programs that process our source code before the process of compilation. There are multiple steps which are involved in the writing and execution of the program. Preprocessor directives start with the '#' symbol. The #define preprocessor is used to create a constant throughout the program. Whenever this name is encountered by the compiler, it is replaced by the actual piece of defined code.

Example:

```
#define long long ll
```

4. Global Declaration

The global declaration section contains global variables, function declaration, and static variables. Variables and functions which are declared in this scope can be used anywhere in the program.

Example:

```
int num = 18;
```

5. Main() Function

Every C program must have a main function. The main() function of the program is written in this section. Operations like declaration and execution are performed inside the curly braces of the main program. The return type of the main() function can be int as well as void too. void() main tells the compiler that the program will not return any value. The int main() tells the compiler that the program will return an integer value.

Example:

```
void main()
```

vtuedge.com

		or **int main()** **6. Sub Programs** **User-defined functions are called in this section of the program. The control of the program is shifted to the called function whenever they are called from the main or outside the main() function. These are specified as per the requirements of the programmer.** **Example:** **int sum(int x, int y)** **{** **return x+y;** **}**		

	b	**Demonstrate formatted output of integer in C with suitable example**
		Demonstrate formatted INPUT of integer in C with suitable example
		
		In C programming, you can use the printf function to display formatted output. To display integers with a specific format, you can use format specifiers
		#include <stdio.h>
		
		int main() {
		int number = 42;
		
		// Displaying the integer with different formats
		printf("Integer: %d\n", number); // %d is the format specifier for integers
		printf("Integer (padded with zeros): %05d\n", number); // Padded with zeros to a width of 5
		printf("Integer (right-aligned): %-10d\n", number); // Right-aligned within a width of 10
		printf("Integer (with sign): %+d\n", number); // Display sign (+ or -)
		
		return 0;
		}
		
		%d is the format specifier for integers.
		
		Demonstrate formatted INPUT of integer in C with suitable example
		
		In C programming, you can use the scanf function to read formatted input, including integers. Here's an example
		
		demonstrating the formatted input of an integer
		#include <stdio.h>
		
		int main() {
		int number;
			L3	**6**

vtuedge.com

```
    // Prompt the user for input
    printf("Enter an integer: ");

    // Read an integer from the user
    if (scanf("%d", &number) == 1) {
        // If the input is a valid integer, display it
        printf("You entered: %d\n", number);
    } else {
        // If the input is not a valid integer, display an error message
        printf("Invalid input. Please enter a valid integer.\n");
    }

    return 0;
}
```

	c	**Discuss different types of error occur in program**	L2	6

Syntax Errors:

- **Definition: Syntax errors occur when the code violates the rules of the programming language.**
- **Examples: Missing semicolons at the end of statements, mismatched parentheses or braces, incorrect use of keywords, etc.**
- **Impact: The program will not compile until syntax errors are fixed. The compiler will generate error messages pointing to the location of the syntax error.**

```
// Example of syntax error
#include <stdio.h>

int main() {
    printf("Hello, world!"  // Missing semicolon
    return 0;
}
```

Logic Errors:

- **Definition: Logic errors occur when the code does not produce the expected output due to mistakes in the algorithm or overall design.**
- **Examples: Incorrect calculations, improper conditional statements, wrong loop conditions, etc.**
- **Impact: The program may compile and run, but it will not produce the desired or correct results. Debugging is required to identify and fix logic errors.**

```
// Example of logic error
#include <stdio.h>

int main() {
    int x = 5, y = 10;
    int sum = x - y;  // Incorrect operation

    printf("Sum: %d\n", sum);
    return 0;
}
```

Runtime Errors:

vtuedge.com

		• Definition: Runtime errors occur while the program is running. They are often related to unexpected conditions or invalid operations during execution.
		• Examples: Division by zero, accessing an array out of bounds, dereferencing a null pointer, etc.
		• Impact: The program may terminate abruptly (crash) or exhibit unpredictable behavior. Identifying and fixing runtime errors often involves debugging tools.
		// Example of runtime error
		#include <stdio.h>
		
		int main() {
		int divisor = 0;
		int result = 10 / divisor; // Division by zero
		
		printf("Result: %d\n", result);
		return 0;
		}		

OR				
Q.02	a	Explain the various rules for forming identifiers names. Give examples for valid and invalid identifiers for the same. In C programming, identifiers are names given to various program elements such as variables, functions, arrays, etc. Identifiers must follow certain rules to be valid. Rules for Forming Identifiers in C: Character Set: • Valid characters for identifiers are letters (both uppercase and lowercase), digits, and the underscore _. • The first character of an identifier must be a letter or an underscore. Length: • Identifiers can be of any length, but only the first 31 characters are significant. Some compilers may support longer identifiers. Keywords: • Identifiers cannot be the same as C keywords (reserved words). For example, int, for, while, etc., are reserved and cannot be used as identifiers. Whitespace: • No whitespace characters are allowed within an identifier.	L2	8

vtuedge.com

		Examples of Valid Identifiers:
		
		int variable; // valid
		float average_score; // valid
		char _symbol; // valid
		int x1; // valid
		
		Examples of Invalid Identifiers:
		
		int 123number; // invalid (starts with a digit)
		double my-score; // invalid (contains a hyphen)
		char first name; // invalid (contains a whitespace)
		float while; // invalid (while is a keyword)		

	b	**Mention various output devices and explain hardcopy devices** **Common Output Devices:** **Monitors (Visual Display Units):** • **Monitors display information in visual form on a screen. They come in various types, such as CRT (Cathode Ray Tube), LCD (Liquid Crystal Display), LED (Light Emitting Diode), and more.** **Printers:** • **Printers produce a hardcopy of the information processed by a computer. There are different types of printers, including dot matrix, inkjet, laser, and thermal printers.** **Plotters:** • **Plotters are used to produce high-quality graphical output. They use pens to draw images on paper based on computer instructions.** **Projectors:** • **Projectors display computer-generated images or presentations on a large screen or surface, often used in classrooms and business presentations.** **Speakers:** • **Speakers provide audio output, allowing users to hear sound produced by the computer, such as music, videos, or system alerts.** **Hardcopy Devices:** **Hardcopy devices produce a permanent, physical copy of the information. The primary hardcopy device is a printer. Printers can be further categorized based on their printing technology:** **Dot Matrix Printers:**	**L1**	**6**

vtuedge.com

		• These printers use a matrix of pins to strike an inked ribbon, creating dots on paper. They are relatively slow but are capable of producing multipart forms. They are used in applications like invoice printing. Inkjet Printers: • Inkjet printers spray tiny droplets of ink onto paper to create images or text. They are suitable for producing high-quality color prints and are commonly used for documents and photo printing.		
	c	**Explain generation of computer** **1. First Generation (1940s-1950s):** • Technology: Vacuum tubes were used for electronic components. • Characteristics: Large, expensive, and prone to overheating. Limited to batch processing. • Examples: ENIAC (Electronic Numerical Integrator and Computer), UNIVAC I. **2. Second Generation (1950s-1960s):** • Technology: Transistors replaced vacuum tubes, leading to smaller and more reliable computers. • Characteristics: Faster, smaller, and more affordable. Introduction of assembly language and high-level programming languages. • Examples: IBM 1401, IBM 7090 series. **3. Third Generation (1960s-1970s):** • Technology: Integrated circuits (ICs) replaced individual transistors, leading to increased computing power. • Characteristics: Smaller, more powerful, and more reliable. Introduction of time-sharing and multiprogramming. • Examples: IBM System/360, CDC 6600. **4. Fourth Generation (1970s-1980s):** • Technology: Microprocessors, VLSI (Very Large Scale Integration) chips, and the development of personal computers. • Characteristics: Smaller, cheaper, and accessible to individuals. Rise of GUIs (Graphical User Interfaces) and networking. • Examples: IBM PC, Apple Macintosh, Commodore 64. **5. Fifth Generation (1980s-Present):** • Technology: Parallel processing, artificial intelligence (AI), and advanced software development. • Characteristics: Focus on parallel processing, AI, and natural language processing. Integration of voice and image recognition. • Examples: IBM Watson, Google's DeepMind.	L2	6

vtuedge.com

Module-2		Module-2		
Q. 03	a	**Demonstrate the functioning of Bitwise operator in C** **Bitwise operators** Bitwise operators are used to manipulate bits in various different ways. They are equivalent to how we use mathmatical operations like (+, -, /, *) among numbers, similarly we use bitwise operators like (|, &, ^, <<, >>, ~) among bits. **Type of Bitwise Operators** **Bitwise AND(&) : It** takes two numbers as operands and does AND on every bit of two numbers. The result of AND is 1 only if both bits are 1. **Bitwise OR(|) :** Ittakes two numbers as operands and does OR on every bit of two numbers. The result of OR is 1 if any of the two bits is 1. **Bitwise XOR (^) :**It takes two numbers as operands and does XOR on every bit of two numbers. The result of XOR is 1 if the two bits are different. **Left shift(<<) : It takes two numbers, the left shifts the bits of the first operand, and the second operand decides the number of places to shift.** **Right shift(>>) : It takes two numbers, right shifts the bits of the first operand, and the second operand decides the number of places to shift.** **Bitwise NOT(~): It takes one number and inverts all bits of it.**	L3	6

	b	**Write a C program to find roots of quadratic equation**
		
		`#include<stdio.h>`
		`#include<stdlib.h>`
		`#include<math.h>`
		`int main()`
		`{`
		`float a, b, c, d,root1,root2,real,imag;`
		`printf("enter the coefficients a, b, c:\n");`
		`scanf("%f%f%f",&a,&b,&c);`
		`if(a==0||b==0||c==0)`
		`{`
		`printf("Invalid coefficients\n");`
		`exit(0);`
		`}`
		`d=b*b-4*a*c;`
		`if(d>0)`
		`{`
		`root1=(-b+sqrt(d))/(2.0*a);`
		`root2=(-b-sqrt(d))/(2.0*a);`
		`printf("the roots are real and distinct...\n");`
		`printf("root1=%.2f\n",root1);`
		`printf("root2=%.2f\n",root2);`
		`}`
		`else if(d==0)`
		`{`
		`root1=root2=-b/(2.0*a);`
		`printf("the roots are real and equal..\n");`
		`printf("root1=%.2f\n",root1);`
			L3	8

vtuedge.com

```
printf("root2=%.2f\n",root2);
}
{
real= -b/(2.0*a);
imag=sqrt(fabs(d))/(2.0*a);
printf("the roots are complex and
imaginary..\n");
printf("root1=%.2f+i%.2f\n",real,imag);
printf("root2=%.2f-i%.2f\n",real,imag);
}
return 0;
}
```

c

Distinguish between the break and continue statement — L4 — 6

break statement: This statement terminates the smallest enclosing loop (i.e., while, do-while, for loop, or switch statement). Below is the program to illustrate the same:

Example:

```
#include <stdio.h>
 int main()
{
   int i = 0, j = 0;

   for (int i = 0; i < 5; i++)
{
printf("i = %d, j = ", i);
  for (int j = 0; j < 5; j++) {

        if (j==2 )
          break;

    printf("%d ", j);
    }
    printf("\n");

  }
 return 0;
}
```

continue statement: This statement skips the rest of the loop statement and starts the next iteration of the loop to take place. Below is the program to illustrate the same.

Example:

```
#include <stdio.h>
 int main()
{
   int i = 0, j = 0;

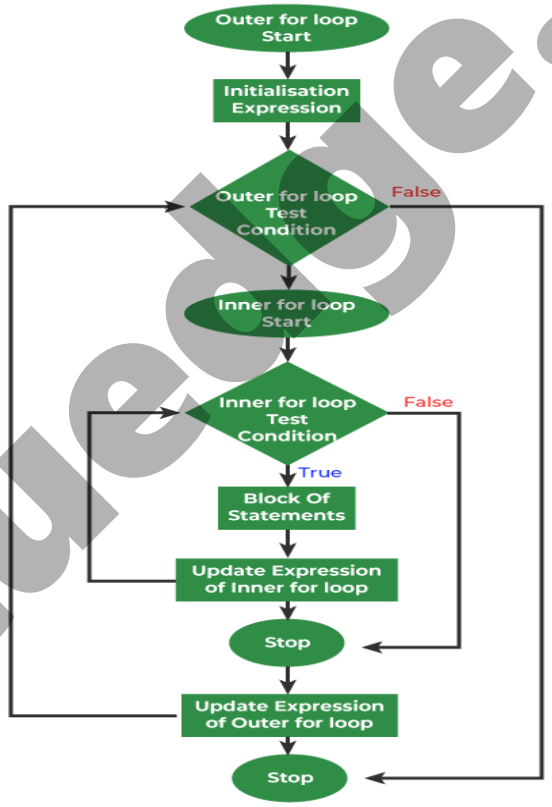
   for (int i = 0; i < 5; i++)
{
printf("i = %d, j = ", i);
    for (int j = 0; j < 5; j++) {
```

vtuedge.com

```
        if (j==2 )
         continue;

    printf("%d ", j);
     }
     printf("\n");

  }
 return 0;
}
```

OR				
Q.04	**a**	**Illustrate Nested loops in C with suitable example** **A nested loop means a loop statement inside another loop statement. That is why nested loops are also called "loop inside loops". We can define any number of loops inside another loop.** **1. Nested for Loop** **Nested for loop refers to any type of loop that is defined inside a 'for' loop. Below is the equivalent flow diagram for nested 'for' loops**  **Syntax:** **for (initialization; condition; increment) {** **for (initialization; condition; increment) {** **// statement of inside loop** **}** **// statement of outer loop** **}**	**L3**	**6**

vtuedge.com

Example :

```
#include <stdio.h>



int main()
{
 int arr[2][3]  =  { { 0, 6 }, { 1, 7 }, { 2, 8 } };

   for (int i = 0; i < 2; ++i) {

      for (int j = 0; j < 3; ++j) {
        printf("Element at arr[%i][%i][%i] = %d\n",  i, j, k, arr[i][j][k]);

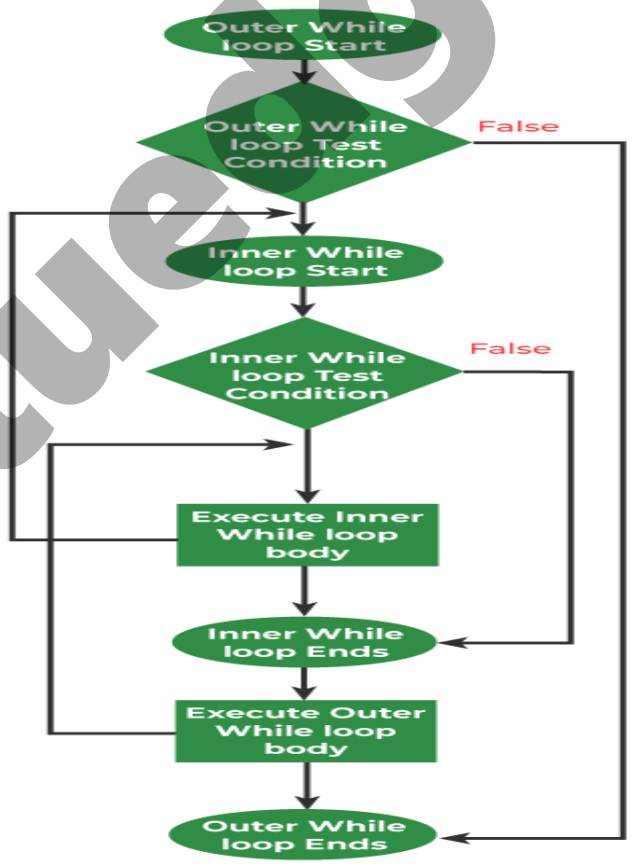
         }

      }

   return 0;
}
```

2. Nested while Loop

A nested while loop refers to any type of loop that is defined inside a 'while' loop. Below is the equivalent flow diagram for nested 'while' loops:



vtuedge.com

Syntax:

```
while(condition) {

  while(condition) {

    // statement of inside loop
  }

  // statement of outer loop
}
```

Example: Print Pattern using nested while loops

```
// C program to print pattern using nested while loops
#include <stdio.h>



int main()
{

   int end = 5;

   printf("Pattern Printing using Nested While loop");

 int i = 1;

   while (i <= end) {

      printf("\n");

      int j = 1;

      while (j <= i) {

         printf("%d ", j);

         j = j + 1;

      }

      i = i + 1;

   }

   return 0;
}
```

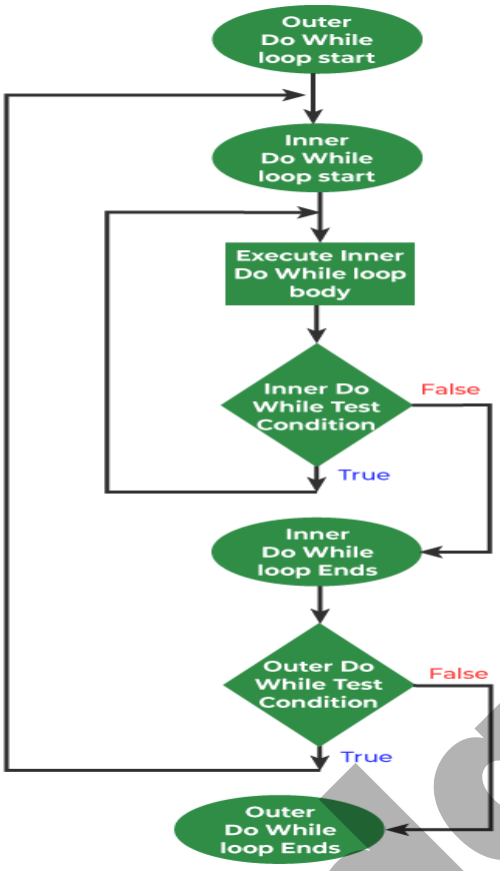
Output

```
Pattern Printing using Nested While loop
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

vtuedge.com

3. Nested do-while Loop

A nested do-while loop refers to any type of loop that is defined inside a do-while loop. Below is the equivalent flow diagram for nested 'do-while' loops:



Syntax:

```
do{

  do{

    // statement of inside loop
  }while(condition);

  // statement of outer loop
}while(condition);
```

Example: Below program uses a nested for loop to print all prime factors of a number.

```
#include <math.h>
#include <stdio.h>


// A function to print all prime factors of a given number n

void primeFactors(int n)
{
```

vtuedge.com

```
    // Print the number of 2s that divide n
    while (n % 2 == 0) {
        printf("%d ", 2);
        n = n / 2;
    }

    for (int i = 3; i <= sqrt(n); i = i + 2) {
        // While i divides n, print i and divide n
        while (n % i == 0) {
            printf("%d ", i);
            n = n / i;
        }
    }

    if (n > 2)
        printf("%d ", n);
}

int main()
{
    int n = 315;
    primeFactors(n);
    return 0;
}
```

	b	**Write a C program to print whether a given number is palindrome or not**	L3	7

```
#include <stdio.h>

int main() {
int n,r,s=0,x;
scanf("%d",&n);
x=n;
while(x!=0)
{
    r=x%10;
    s=s*10+r;
    x/=10;
```

vtuedge.com

```
}
printf("Reverse of the number is %d",s);
if(s==n)
printf("\nNumber is a palindrome");
else
printf("\nNumber is not a palindrome");
   return 0;
}
```

	c	**Explain switch statements with syntax. Write a C program to simulate calculator** **Switch Statement** **The switch statement in C is an alternate to if-else-if ladder statement which allows us to execute multiple operations for the different possibles values of a single variable called switch variable.Here, We can define various statements in the multiple cases for the different values of a single variable.**	**L3**	**7**

Syntax:

```
switch(expression){
case value1:
 //code to be executed;
 break;  //optional
case value2:
 //code to be executed;
 break;  //optional
.....

default:
 code to be executed if all cases are not matched;
}
```

Example :

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
int a, b, res; char op;
printf("Enter a simple arithmetic expression:\n");
scanf("%d%c%d",&a,&op,&b);
switch(op)
{
case '+':res=a+b;
break;
case '-':res=a-b;
break;
case '*':res=a*b;
break;
case '/':if(b!=0)
res=a/b;
else
{
printf("division by zero is not possible\n");
exit(0);
}
```

vtuedge.com

		break;
		case '%':res=a%b
		; break;
		default:printf("Illigal operator\n");
		exit(0);
		}
		printf("\nResult=%d",res);
		return 0;
		}		

Module-3				

Q. 05	a	Write a C program to implement Bubble sort technique(ascending order)
		Sorting elements in ascending order using bubble sort
		#include<stdio.h>
		#include<conio.h>
		void main()
		{
		int i,j,a[10],temp;
		clrscr();
		printf(" Enter the number of elements:");
		scanf("%d",&n);
		printf("Enter the elements into an array\n");
		for(i=0;i<n;i++);
		{ scanf("%d",a[i]);
		}
		for(i=0;i<n-1;i++)
		{
		for(j=0;j<n-1-i;j++)
		{
		if(a[j]>a[j+1])
		{
		temp=a[j];
		a[j]=a[j+1];
		a[j+1]=temp;
		}
		}}
		printf(" The sorted elements are \n");
		for(i=0;i<n;i++)
		{
		printf("%d",a[i]");
		}
		return 0;
		getch();
		}
			L3	8

	b	Illustrate the concept of recursive function with example A recursive function is a function that calls itself directly or indirectly. It is one of the most important fundamental concepts in c programming. It helps to break down a complex problem into smaller sub- problems. Here's an example c program that uses a recursive function to find the factorial of a number	L3	6

vtuedge.com

		#include<stdio.h>
		#include<conio.h>
		void main()
		{
		Int n;
		clrscr();
		printf("Enter the number:");
		scanf("%d",&n);
		if(n<0)
		printf("invalid number");
		else
		return 1;
		else
		return(x*fact(x-1));
		}
		
		Output:
		Enter the number: 5
		5!=120		

	c	**Discuss various scope of variables**
		The scope is the variable region in which it can be used. Beyond that area, we cannot use a variable. The local and global are two scopes for c variables.
		Local scope:
		The local scope is limited to the code or function in which the variable is declared.
		Global scope :
		Global scope is the entire program. Global variables can be used anywhere throughout the program.
		There are 2 types of variables ont he basis of Scope.
		Local variables:
		The variable that is declared in a function or code is called a local variable.
		The scope of local variables is within the defined function only. We cannot use a local variable outside the function(in which it is declared).
		Global variables:
		The scope of the global variable is the entire program. They are not defined inside any function and can be used in any function.
		Example program:
		#include<stdio.h>
		char name= surendra; //Declaring global variable//
		void person()
		{
		int age=20; //Declaring local variable//
		Float height=5.8;
		printf(" age is%d\n, age);
		printf("height is %f",height);
		printf(" name is %c",name);
		}
		int main()
		{
		person();
			L2	6

vtuedge.com

		return 0; }		
OR				

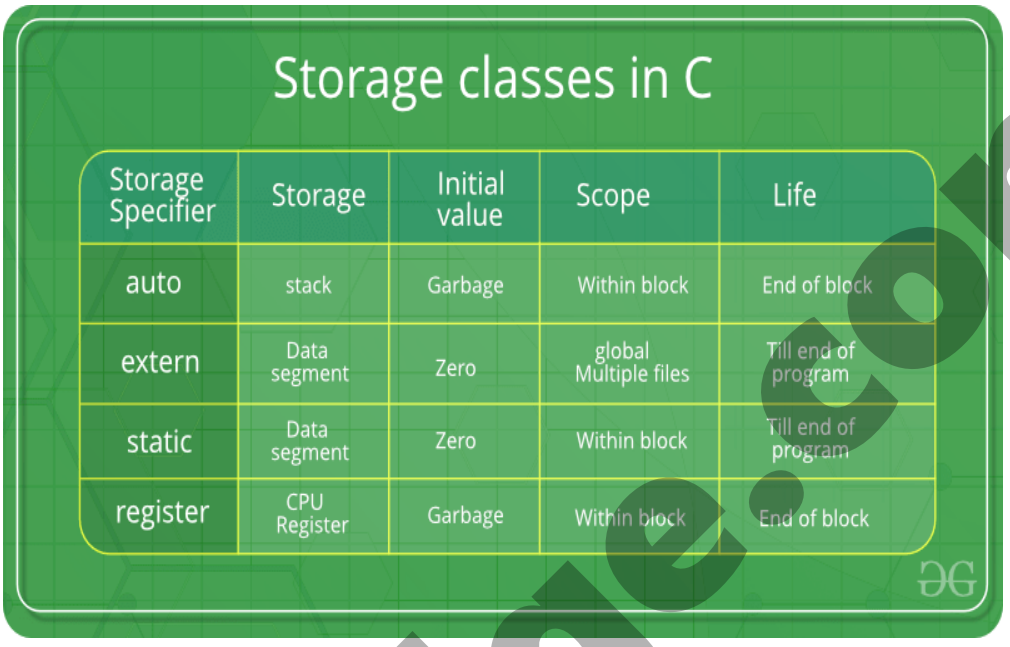
Q. 06	a	**Differentiate between call by value and call by reference. Using suitable example**
		Call by value:
		• The value of the actual parameters is copied into the formal parameters
		• The arguments in the function call(Actual arguments) are not modified by the change in parameters of the called function(formal arguments)
		• Different memory is allocated for actual and formal parameters.
		Example : program to show call by value.
		#include<stdio.h>
		#include<conio.h>
		void main()
		{
		int a,b,swap();
		clrscr();
		a=5;
		b=10;
		printf(" value of a=%d and value of b=%d before swap",a,b);
		swap(a,b);
		printf("\n value of a=%d and b=%d after swap",a,b);
		getch();
		}
		int swap(int x,int y)
		{
		int temp;
		temp=x;
		x=y;
		y=temp;
		}
		
		Call by reference:
		• The address of the variable is passed into the function call as the actual parameter
		• The value of the actual parameters can be modified by changing the formal parameters since the address of the actual parameters is passed
		• Same memory is allocated for both formal parameters.
		Example: program to show call by reference.
		#include<stdio.h>
		#include<conio.h>
		void main()
		{
		int a,b,*aa,*bb,swap();
		clrscr();
		a=5;
		b=10;
		aa=&a;
		bb=&b;
		printf("value of a=%d and value of b=%d before swap",a,b);
			L4	8

vtuedge.com

		swap(aa,bb);
		printf("\n value of a=%d and b=%d after swap",a,b);
		getch();
		}
		int swap(int *x,int*y)
		{
		int temp;
		temp=*x;
		*x=*y;
		*y=temp;
		}		

	b	**Write a C program to transpose a MxN matrix**
		#include<stdio.h>
		#include<conio.h>
		void main()
		{
		Int a[3][2],b[2][3],i,j;
		clrscr();
		printf("Enter values of matrix:");
		for(i=0;i<3;i++)
		{
		for(j=0;j<2;j++)
		scanf("%d",&a[i][j]);
		}
		printf("Matrix:\n");
		for(i=0;i<3;i++)
		{
		for(j=0;j<2;j++)
		printf("%d",a[i][j]);
		printf("\n");
		}
		for(i=0;i<3;i++)
		{
		b[j][i]=a[i][j];
		}
		printf("Transpose matrix:\n");
		for(i=0;i<2;i++)
		{
		for(j=0;j<3;j++)
		printf("%d",b[i][j]);
		printf("\n");
		}
		getch();
		}
		
		Output:
		Enter values of the matrix: 4
		5
		6
		1
		2
			L3	8

vtuedge.com

		3 **Matrix:** **4 5** **61** **23** **Transpose matrix:** **4 6 2** **5 1 3**		
	c	**Discuss the various storage classes** **C language uses 4 storage classes, namely:**  **1. auto** **This is the default storage class for all the variables declared inside a function or a block. Hence, the keyword auto is rarely used while writing programs in C language. Auto variables can be only accessed within the block/function they have been declared and not outside them (which defines their scope). Of course, these can be accessed within nested blocks within the parent block/function in which the auto variable was declared.** **However, they can be accessed outside their scope as well using the concept of pointers given here by pointing to the very exact memory location where the variables reside. They are assigned a garbage value by default whenever they are declared.** **2. extern** **Extern storage class simply tells us that the variable is defined elsewhere and not within the same block where it is used. Basically, the value is assigned to it in a different block and this can be overwritten/changed in a different block as well. So an extern variable is nothing but a global variable initialized with a legal value where it is declared in order to be used elsewhere. It can be accessed within any function/block.** **Also, a normal global variable can be made extern as well by placing the 'extern' keyword before its declaration/definition in any function/block. This basically signifies that we are not initializing a new variable but instead, we are using/accessing the global variable only. The main purpose of using extern variables is that they can be accessed between two different files which are part of a large program.**	**L2**	**4**

vtuedge.com

		3. static **This storage class is used to declare static variables which are popularly used while writing programs in C language. Static variables have the property of preserving their value even after they are out of their scope! Hence, static variables preserve the value of their last use in their scope. So we can say that they are initialized only once and exist till the termination of the program. Thus, no new memory is allocated because they are not re-declared.** **Their scope is local to the function to which they were defined. Global static variables can be accessed anywhere in the program. By default, they are assigned the value 0 by the compiler.** **4. register** **This storage class declares register variables that have the same functionality as that of the auto variables. The only difference is that the compiler tries to store these variables in the register of the microprocessor if a free register is available. This makes the use of register variables to be much faster than that of the variables stored in the memory during the runtime of the program.** **If a free registration is not available, these are then stored in the memory only. Usually, a few variables which are to be accessed very frequently in a program are declared with the register keyword which improves the running time of the program. An important and interesting point to be noted here is that we cannot obtain the address of a register variable using pointers.**		
		Module-4		
Q. 07	**a**	**Mention various operations that can be performed on string using built-in functions. Explain any two function** **In C programming, you can perform various operations on strings using standard library functions from the `string.h` header. Here are two commonly used string operations:** **1. **String Length (strlen):**** **- **Description:** The `strlen` function returns the length (number of characters) of a string excluding the null terminator.** **- **Example:**** **```c** **#include <stdio.h>** **#include <string.h>** **int main() {** **char myString[] = "Hello, world!";**	**L2**	**8**

vtuedge.com

		int length = strlen(myString);
		printf("Length of the string: %d\n", length);
		return 0;
		}
		``
		Output: `Length of the string: 13`
		
		2. **String Copy (strcpy):**
		- **Description:** The `strcpy` function is used to copy one string to another. It copies the characters from the source string to the destination string until it encounters the null terminator.
		- **Example:**
		``c
		#include <stdio.h>
		#include <string.h>
		
		int main() {
		char source[] = "Hello, world!";
		char destination[20]; // Ensure the destination buffer is large enough
		
		strcpy(destination, source);
		printf("Copied string: %s\n", destination);
		return 0;
		}
		``
		Output: `Copied string: Hello, world!`
		
		These functions are essential for working with strings in C, providing capabilities such as determining string length and copying strings.		

	b	Develop a program using pointer to compute the sum, mean and standard deviation of all element stored in array of N real number Here's a C program that uses pointers to compute the sum, mean, and standard deviation of an array of N real numbers: ```c #include <stdio.h> #include <math.h>	L4	8

vtuedge.com

```
void calculateStats(double *arr, int size, double *sum, double *mean,
double *stdDev) {
    // Calculate sum
    *sum = 0;
    for (int i = 0; i < size; i++) {
        *sum += arr[i];
    }

    // Calculate mean
    *mean = *sum / size;

    // Calculate standard deviation
    *stdDev = 0;
    for (int i = 0; i < size; i++) {
        *stdDev += pow(arr[i] - *mean, 2);
    }
    *stdDev = sqrt(*stdDev / size);
}

int main() {
    int size;
    printf("Enter the number of elements: ");
    scanf("%d", &size);

    double numbers[size];

    printf("Enter %d real numbers:\n", size);
    for (int i = 0; i < size; i++) {
        scanf("%lf", &numbers[i]);
    }

    double sum, mean, stdDev;
    calculateStats(numbers, size, &sum, &mean, &stdDev);

    printf("Sum: %.2f\n", sum);
    printf("Mean: %.2f\n", mean);
    printf("Standard Deviation: %.2f\n", stdDev);

    return 0;
}
```

This program prompts the user to enter the number of elements and then takes the input for each real number. It calculates the sum, mean, and standard deviation using pointers and prints the results. The `%lf` format specifier is used for reading and printing double values.

vtuedge.com

	c	**Explain how strings are represented in main memory** Strings are objects representing a sequence of character values. An array of characters works the same as a string. So, a string is an array of characters or an object used to represent a sequence of	L2	4

characters. A string is referred to as using a character pointer or a character array.

Storing Strings as Character Arrays

The strings declared as character arrays are stored like other arrays in C. For example, if str[] is an auto variable, the string is stored in the stack segment; if it's a global or static variable, then stored in the data segment.

Strings as character arrays can be stored in two ways:

```
char str[6] = "Ninja"; /*One extra for string terminator*/
char str[6] = {'N','i', 'n', 'j', 'a', '\0'}; /* '\0' is string terminator */
```

Storing Strings as Character Pointers

Strings can be stored using character pointers in two ways:

Read-only string in a shared segment.

The directly assigned string values to a pointer are stored in a read-only block shared among functions in most compilers.

```
char *str = "Ninja";
```

The word "Ninja" is stored in a shared read-only location while the pointer str is stored in read-write memory. The pointer str can be changed to point to something else, but it cannot change the value at the present str. So this kind of string should be used only when the string is not modified at a later stage in the program.

Dynamically allocated in the heap segment.

Strings are stored like other dynamically allocated things in C and shared among functions.

```
char *str;
int size = 6; /*one extra for '\0'*/
str = (char *)malloc(sizeof(char)*size);
*(str+0) = 'N';
*(str+1) = 'i';
*(str+2) = 'n';
*(str+3) = 'j';
*(str+4) = 'a';
*(str+5) = '\0';
```

Example (modify a string)

```
int main()
{
 char str[] = "Ninja";
 *(str+1) = 'n';
```

Vtuedge.com

		getchar();
		return 0;
		}		

OR				

Q. 08	**a**	**Write a program to compare two strings without using built-in function**
		
		int main()
		{
		char s[20] = “college”;
		char d[20] = “terna”;
		int i = 0;
		while(s[i] != ‘\0’ || d[i] != ‘\0’)
		{
		if(d[i] == s[i])
		{
		cmp = 0;
		i++;
		}
		else
		{
		cmp = 1;
		break;
		}
		}
		f(cmp == 0)
		printf(“Strings are equal);
		else
		printf(“Strings are different”);
		return 0;
		}
			L3	**8**

	b	**What is a pointer? Discuss pointer arithmetic with suitable C code** **Pointers are one of the core components of the C programming language. A pointer can be used to store the memory address of other variables, functions, or even other pointers. The use of pointers allows low-level memory access, dynamic memory allocation, and many other functionality in C.** **A pointer in c is an address, which is a numeric value. Therefore, you can perform arithmetic operations on a pointer just as you can on a numeric value. There are four arithmetic operators that can be used on pointers: ++, --, +, and -.** **Incrementing a Pointer** **Syntax :**	**L2**	**6**

vtuedge.com

```
data_type *ptr;
ptr = /* initialize ptr to point to some memory location */;

// Increment the pointer to the next element
ptr++;
```

Example:

```
#include <stdio.h>
int main(){
int numbers[] = {10, 20, 30, 40, 50};
int *ptr = numbers; // Point to the first element
ptr++;  // Increment the pointer to move to the next element
return 0;
}          // Now ptr points to the second element (20)
```

Decrementing a Pointer

Decrementing a pointer is identical to incrementing it, except it transfers it to the prior memory address.

Syntax :

```
data_type *ptr;
ptr = / * initialize ptr to point to some memory location */;

// Decrement the pointer to the previous element
ptr--;
```

Example:

```
#include <stdio.h>
int main(){
int numbers[] = {10, 20, 30, 40, 50};
int *ptr = numbers;
ptr--;
return 0;
}
```

Addition of Integer to a Pointer

Adding an integer value to a pointer is a basic operation with major ramifications for memory management and data traversal.

Example:

```
// C program to illustrate pointer Addition
#include <stdio.h>

// Driver Code
int main()
{
  // Integer variable //
   int N = 4;
```

vtuedge.com

```
    // Pointer to an integer
    int *ptr1, *ptr2;

    // Pointer stores the address of N
    ptr1 = &N;
    ptr2 = &N;

    printf("Pointer ptr2 before Addition: ");
    printf("%p \n", ptr2);

    // Addition of 3 to ptr2
    ptr2 = ptr2 + 3;
    printf("Pointer ptr2 after Addition: ");
    printf("%p \n", ptr2);

    return 0;
}
```

Output

```
Pointer ptr2 before Addition: 0x7ffca373da9c
Pointer ptr2 after Addition: 0x7ffca373daa8
```

Subtraction of Integer from a Pointer

Subtracting an integer from a pointer essentially pushes the pointer backwards in memory by the computed offset. This can be handy for exploring arrays or controlling dynamic memory allocation.

```
// C program to illustrate pointer Subtraction
#include <stdio.h>

// Driver Code
int main()
{
    // Integer variable
    int N = 4;

    // Pointer to an integer
    int *ptr1, *ptr2;

    // Pointer stores the address of N
    ptr1 = &N;
    ptr2 = &N;

    printf("Pointer ptr2 before Subtraction: ");
    printf("%p \n", ptr2);

    // Subtraction of 3 to ptr2
    ptr2 = ptr2 - 3;
    printf("Pointer ptr2 after Subtraction: ");
    printf("%p \n", ptr2);

    return 0;
```

vtuedge.com

		} **Output** Pointer ptr2 before Subtraction: 0x7ffd718ffebc Pointer ptr2 after Subtraction: 0x7ffd718ffeb0		

	c	**Explain gets()and puts() function with example**
		
		The gets() and puts() are declared in the header file stdio.h. Both the functions are involved in the input/output operations of the strings.
		
		C gets() function
		
		The gets() function enables the user to enter some characters followed by the enter key. All the characters entered by the user get stored in a character array. The null character is added to the array to make it a string. The gets() allows the user to enter the space-separated strings. It returns the string entered by the user.
		Declaration
		
		char[] gets(char[]);
		
		Reading string using gets()
		
		#include<stdio.h>
		
		void main ()
		
		{
		
		char s[30];
		
		printf("Enter the string? ");
		
		gets(s);
		
		printf("You entered %s",s);
		
		}
		
		Output
		Enter the string?
		How are you doing?
		You entered How are you doing?
		
		C puts() function
		
		The puts() function is used to print the string on the console which is previously read by using gets() or scanf() function. The puts() function returns an integer value representing the number of characters being printed on the console. Since, it prints an additional newline character with the string, which moves the cursor to the new line on the console, the integer value returned by puts() will always be equal to the number of characters present in the string plus 1.
			L2	6

vtuedge.com

Declaration:

```
int puts(char[])
```

Example:

```
#include<stdio.h>
#include <string.h>
int main(){
char name[50];
printf("Enter your name: ");
gets(name); //reads string from user
printf("Your name is: ");
puts(name);  //displays string
return 0;
}
```

Output:

```
Enter your name: chaitanya
Your name is: chaitanya
```

Module-5				
Q. 09	**a**	**Explain various modes in which file can be opened for processing** **File Opening Modes** **Mode** / **Meaning** / **Description** **r** / **Read** / **Only reading possible. No, create the file if it does not exist.** **w** / **Write** / **Only writing is possible. Create the file if it does not exist; otherwise, erase the old content of the file and open a blank file.** **a** / **Append** / **Only writing is possible. Create a file; if it does not exist, otherwise open the file and write from the end of the file. (Do not erase the old content).**	**L2**	**7**

vtuedge.com

		r+ Reading + Writing: Reading and writing are possible. Create a file if it does not exist, overwriting existing data. Used for modifying content.
		
		w + Reading + Writing: Reading and writing are possible. Create a file if it does not exist. Erase old content.
		
		a+ Reading + Appending: Reading and writing are possible. Create a file if it does not exist. Append content at the end of the file.
		
		EXAMPLE
		`#include <stdio.h>`
		
		`int main (){`
		
		`FILE *fp;`
		`fp= fopen (" myfile.dat " ," r ");  // file opening mode.`
		`if (fp == NULL)  {`
		
		`printf (" File cannot open. ");`
		`}`
		`return 0 ;`
		`}`		

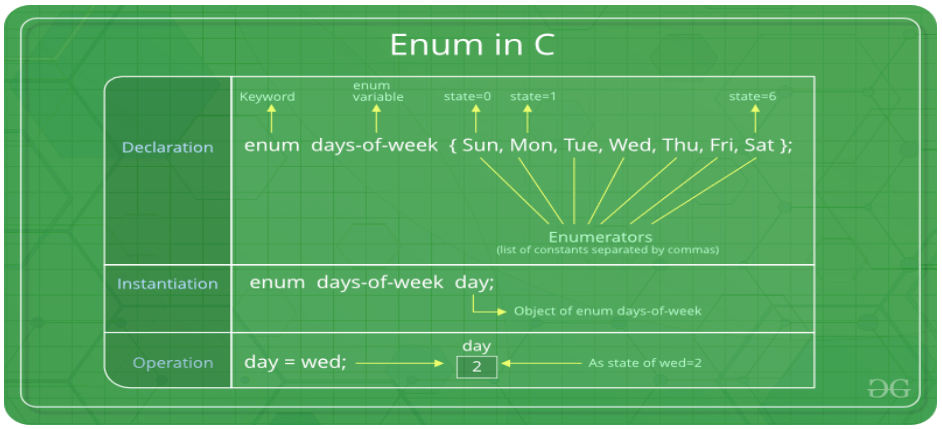
	b	**Implement structure to read, write and compute average marks of the students. List the students scoring above and below the average marks for a class of n students**
		
		`#include<stdio.h>`
		`#include<string.h>`
		`struct student {`
		`int rollno;`
		`char name[20];`
		`float marks;`
		`};`
		`int main()`
		`{`
		`int i,n,found=0;`
		`struct student s[10];`
		`float sum=0,average;`
		`printf("\nEnter the number of student details");`
		`scanf("%d",&n);`
		`for(i=0; i<n; i++) {`
			L3	8

vtuedge.com

```
printf("\nenter the %d student details",i+1);
printf("\n enter roll number:");
scanf("%d",&s[i].rollno);
printf("\n enter student name");
scanf("%s",s[i].name);
printf("\n enter the marks:");
scanf("%f",&s[i].marks);
}
printf("\nStudent details are\n");
printf("\nRollno\t\tName\t\tMarks\n");
for(i=0; i<n; i++)
printf("%d\t\t%s\t\t%f\n",s[i].rollno,s[i].name,s[i].marks);
for(i=0; i<n; i++)
sum=sum+s[i].marks;
average=sum/n;
printf("\nAVERAGE=%,2f",average);
printf("\n students scoring above average\n");
for(i=0; i<n; i++) {
if(s[i].marks>=average) {
printf("%s\t",s[i].name);
}
}
printf("\n students scoring below average\n");
for(i=0; i<n; i++) {
if(s[i].marks<average) {
printf("%s\t",s[i].name);
found=1;
}
}
if(found==0)
printf("\n no students scored below average");
return 0;
}
```

	c	**What are enumeration variable? How are they declared** **Enumeration (or enum) is a user defined data type in C. It is mainly used to assign names to integral constants, the names make a program easy to read and maintain.**	**L1**	**5**

vtuedge.com



enum State {Working = 1, Failed = 0};

The keyword 'enum' is used to declare new enumeration types in C

>>example of enum declaration>>

```
/*The name of enumeration is "flag" and the constant
are the values of the flag. By default, the values of the constants are
as follows:
constant1 = 0, constant2 = 1, constant3 = 2 and so on.*/

enum flag{constant1, constant2, constant3, ..... };
```

Variables of type enum can also be defined. They can be defined in two ways:

```
// In both of the below cases, "day" is
// defined as the variable of type week.

enum week{Mon, Tue, Wed};
enum week day;

// Or

enum week{Mon, Tue, Wed}day;
```

>>PROGRAM EXAMPLE>>

```
// An example program to demonstrate working
// of enum in C
#include<stdio.h>

enum week{Mon, Tue, Wed, Thur, Fri, Sat, Sun};

int main()
{
   enum week day;
   day = Wed;
   printf("%d",day);
   return 0;
}
```

vtuedge.com

		Output: **2**		
OR				
Q. 10	**a**	**Write a short note on functions used to** **Read data from a file** **There are three different functions dedicated to reading data from a file** ● **fgetc(file_pointer): It returns the next character from the file pointed to by the file pointer. When the end of the file has been reached, the EOF is sent back.** ● **fgets(buffer, n, file_pointer): It reads n-1 characters from the file and stores the string in a buffer in which the NULL character '\0' is appended as the last character.** ● **fscanf(file_pointer, conversion_specifiers, variable_adresses): It is used to parse and analyze data. It reads characters from the file and assigns the input to a list of variable pointers variable_adresses using conversion specifiers. Keep in mind that as with scanf, fscanf stops reading a string when space or newline is encountered.** **Write data to a file** **In C, when you write to a file, newline characters '\n' must be explicitly added.** **The stdio library offers the necessary functions to write to a file:** ● **fputc(char, file_pointer): It writes a character to the file pointed to by file_pointer.** ● **fputs(str, file_pointer): It writes a string to the file pointed to by file_pointer.** ● **fprintf(file_pointer, str, variable_lists): It prints a string to the file pointed to by file_pointer. The string can optionally include format specifiers and a list of variables variable_lists.**	**L2**	**8**
	b	**Differentiate structures and unions with syntax and example**	**L4**	**6**

vtuedge.com

	STRUCTURE	UNION
Keyword	The keyword **struct** is used to define a structure	The keyword **union** is used to define a union.
Size	When a variable is associated with a structure, the compiler allocates the memory for each member. The size of structure is **greater than or equal to the sum of sizes of its members.**	when a variable is associated with a union, the compiler allocates the memory by considering the size of the largest memory. So, size of **union is equal to the size of largest member.**
Memory	Each member within a structure is assigned unique storage area of location.	Memory allocated is shared by individual members of union.
Value Altering	Altering the value of a member will not affect other members of the structure.	Altering the value of any of the member will alter other member values.
Accessing members	Individual member can be accessed at a time.	Only one member can be accessed at a time.
Initialization of Members	Several members of a structure can initialize at once.	Only the first member of a union can be initialized.

Syntax

The syntax of structures in C language is,

```
struct structure_name {
   member definition;
} structure_variables;
```

Where,

- **structure_name is the name given to the structure.**
- **member definition is the set of member variables.**
- **structure_variable is the object of structure.**

Example

```
struct Data {
   int a;
   long int b;
} data, data1;
```

Syntax

The syntax of unions in C language is,

```
union union_name {
   member definition;
} union_variables;
```

Where,

- **union_name is any name given to the union.**
- **member definition is the set of member variables.**
- **union_variable is the object of union.**

Example

```
union Data {
   int i;
   float f;
} data, data1;
```

vtuedge.com

	c	**How to detect END-OF-FILE**	L2	6

The function feof() is used to check the end of file after EOF. It tests the end of file indicator. It returns non-zero value if successful otherwise, zero.

Here is the syntax of feof() in C language,

```
int feof(FILE *stream)
```

Here is an example of feof() in C language,

Let's say we have "new.txt" file with the following content −

```
This is demo!
This is demo!
```

Now, let us see the example.

Example

```
#include <stdio.h>
int main() {
   FILE *f = fopen("new.txt", "r");
   int c = getc(f);
   while (c != EOF) {
      putchar(c);
      c = getc(f);
   }
   if (feof(f))
   printf("
Reached to the end of file.");
   else
   printf("
Failure.");
   fclose(f);
   getchar();
   return 0;
}
```

Output

```
This is demo!
This is demo!
Reached to the end of file.
```

vtuedge.com

Table showing the Bloom's Taxonomy level, course outcome and program outcomes				
Questions		Bloom's Taxonomy	Course Outcome	Program Outcomes
Q. 1	a	L2	CO1	PO1,PO2
	b	L3	CO2	PO1,PO2
	c	L2	CO2	PO1,PO2
Q.2	a	L2	CO2	PO1,PO2

	b	L1	CO1	PO1
	c	L2	CO1	PO1
Q.3	a	L3	CO2	PO1,PO2
	b	L3	CO2,CO5	PO1,PO2,PO3
	c	L4	CO2	PO1,PO2
Q.4	a	L3	CO2	PO1,PO2
	b	L3	CO2,CO5	PO1,PO2,PO3
	c	L3	CO2,CO5	PO1,PO2
Q.5	a	L3	CO3	PO1,PO2, PO3
	b	L3	CO3,CO5	PO1,PO2,PO3
	c	L2	CO2	PO1,PO2
Q.6	a	L4	CO4	PO1,PO2,PO3
	b	L3	CO3	PO1,PO2,PO3
	c	L2	CO2	PO1,PO2
Q.7	a	L2	CO5	PO1,PO2, PO3
	b	L4	CO4	PO1,PO2,PO3
	c	L2	CO2	PO1
Q.8	a	L3	CO5	PO1,PO2,PO3
	b	L2	CO4	PO1,PO2
	c	L2	CO2	PO1

Q.9	a	L2	CO5	PO1,PO2
	b	L3	CO4,CO5	PO1,PO2,PO3
	c	L1	CO4	PO1
Q.10	a	L2	CO2,CO5	PO1,PO2
	b	L4	CO4	PO1,PO2
	c	L2	CO2	PO1

vtuedge.com